

DIWAKAR EDUCATION HUB

DISCRETE STRUCTURES AND OPTIMIZATION UNIT - 1

As per updated syllabus

DIWAKAR EDUCATION HUB

THE LEARN WITH EXPERTIES

Propositional Logic

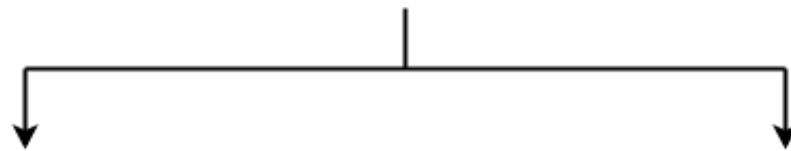
The rules of logic give precise meaning to mathematical statements. These rules are used to distinguish between valid and invalid mathematical arguments.

Propositions

A proposition is a declarative sentence that is either true or false, but not both.

- Propositional logic the area of logic that deals with propositions.
- Propositional variables that represent propositions: p, q, r, s .
- Truth values T, F
- Proposition is a declarative statement that is either true or false but not both.
- Connectives are used to combine the propositions.

Types of Propositions



Atomic Propositions

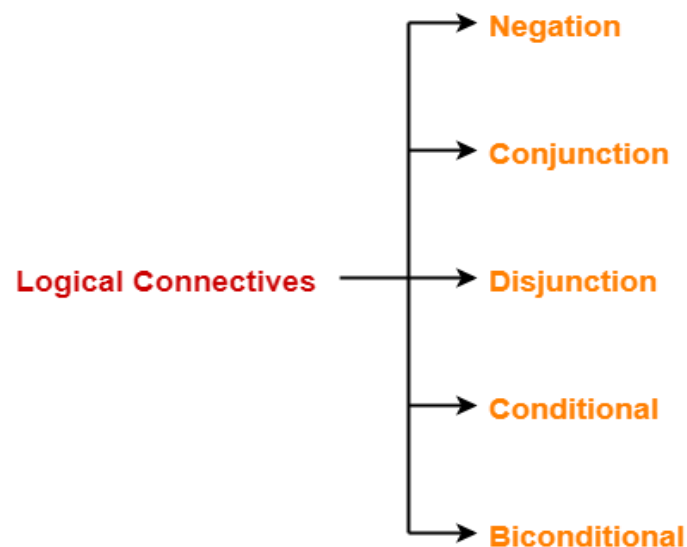
Compound Propositions

About Connectives in Propositional logic-

Logical Connectives-

Connectives are the operators that are used to combine one or more propositions.

In propositional logic, there are 5 basic connectives-



Name of Connective	Connective Word	Symbol
Negation	Not	$\neg$ or $\sim$ or $'$ or $-$
Conjunction	And	$\wedge$
Disjunction	Or	$\vee$
Conditional	If-then	$\rightarrow$
Bi-conditional	If and only if	$\leftrightarrow$

Negation-

If p is a proposition, then negation of p is a proposition which is-

- True when p is false
- False when p is true.

Truth Table-

p	$\sim p$
F	T
T	F

Example-

If p : It is raining outside.

Then, Negation of p is-

$\sim p$: It is not raining outside.

Conjunction-

If p and q are two propositions, then conjunction of p and q is a proposition which is-

- True when both p and q are true
- False when both p and q are false

Truth Table-

p	q	$p \wedge q$
F	F	F
F	T	F
T	F	F
T	T	T

Example-

If p and q are two propositions where-

- p : $2 + 4 = 6$
- q : It is raining outside.

Then, conjunction of p and q is-

$p \wedge q$: $2 + 4 = 6$ and it is raining outside

3. Disjunction-

If p and q are two propositions, then disjunction of p and q is a proposition which is-

- True when either one of p or q or both are true
- False when both p and q are false

Truth Table-

p	q	$p \vee q$
F	F	F

F	T	T
T	F	T
T	T	T

Example-

If p and q are two propositions where-

- $p : 2 + 4 = 6$
- $q : \text{It is raining outside}$

Then, disjunction of p and q is-

$p \vee q : 2 + 4 = 6$ or it is raining outside

Conditional-

If p and q are two propositions, then-

- Proposition of the type “If p then q” is called a conditional or implication proposition.
- It is true when both p and q are true or when p is false.
- It is false when p is true and q is false.

Truth Table-

p	q	$p \rightarrow q$
F	F	T
F	T	T
T	F	F
T	T	T

Examples-

If $a = b$ and $b = c$ then $a = c$.

- If I will go to Australia, then I will earn more money.

Biconditional-

If p and q are two propositions, then-

- Proposition of the type “p if and only if q” is called a biconditional or bi-implication proposition.
- It is true when either both p and q are true or both p and q are false.
- It is false in all other cases.

Truth Table-

p	q	$p \leftrightarrow q$
F	F	T
F	T	F
T	F	F
T	T	T

Examples-

- He goes to play a match if and only if it does not rain.
- Birds fly if and only if sky is clear.

The common binary connectives are:

$p \wedge q$	Conjunction	p and q
$p \vee q$	Disjunction	p or q
$p \rightarrow q$	Implication	if p then q
$p \leftrightarrow q$	Bi-conditional	p if and only if q
$p \oplus q$	Exclusive or	p x-or q
$p \downarrow q$	not or	p nor q
$p \mid q$ or $p \uparrow q$	not and	p n-and q

In the conditional $p \rightarrow q$, p is the **antecedent** and q is the consequent. The conditional $p \rightarrow q$ is often read informally as “ p implies q ”.

Converse, Contra-positive and Inverse

- The proposition $q \rightarrow p$ is called the **converse** of $p \rightarrow q$. The **contra-positive** of $p \rightarrow q$ is the proposition $\neg q \rightarrow \neg p$. The proposition $\neg p \rightarrow \neg q$ is called the inverse of $p \rightarrow q$.
- We will see that of these three conditional statements formed from $p \rightarrow q$, only the **contra-positive** always has the same truth value as $p \rightarrow q$.

EXAMPLE: What are the **contra-positive**, the converse, and the inverse of the conditional-statement? “**The home team wins whenever it is raining?**”

Solution:

“If it is raining, then the home team wins.”

The **contra-positive** of this conditional statement is

“If the home team does not win, then it is not raining.”

The **converse** is - “If the home team wins, then it is raining.”

The **inverse** is - “If it is not raining, then the home team does not win.”

Bi-conditionals

DEFINITION 4

Let p and q be propositions. The **bi-conditional** statement $p \leftrightarrow q$ is the proposition “ p if and only if q .” The **bi-conditional** statement $p \leftrightarrow q$ is true when p and q have the same truth values, and is false otherwise. **Bi-conditional** statements are also called bi-implications.

The Truth Table for the Bi-conditional $p \leftrightarrow q$

Truth Table for the Bi-conditional $p \leftrightarrow q$

p	q	$p \leftrightarrow q$
T	T	T
T	F	F
F	T	F
F	F	T

The statement $p \leftrightarrow q$ is true when both the conditional statements are $p \rightarrow q$ and $q \rightarrow p$ are true and is false otherwise.

Example: Let p be the statement “You can take the flight,” and let q be the statement “You buy a ticket.” Then $p \leftrightarrow q$ is the statement

Answer “You can take the flight if and only if you buy a ticket.”

Precedence of Logical Operator

Operator precedence is an ordering of logical operators designed to allow the dropping of parenthesis in logical expression.

Precedence of Logical Operators

Operator	Precedence
----------	------------

$\neg$	1
--------	---

$\wedge$	2
----------	---

$\vee$	3
--------	---

$\rightarrow$	4
---------------	---

$\leftrightarrow$	5
-------------------	---

Logic and Bit Operations

- A bit is a symbol with two possible values, 0 (zero) and 1 (one).
- A bit can be used to represent a truth value, because there are two truth values, true and false.
- A variable is called a Boolean variable if its value is either true or false.

Truth Value	Bit	Bit Operators OR, AND, and XOR				
		x	y	$x \vee y$	$x \wedge y$	$x \oplus y$
T	1	0	0	0	0	0
F	0	0	1	1	0	1
		1	0	1	0	1
		1	1	1	1	0

DEFINITION

A bit string is a sequence of zero or more bits.

The length of this string is the number of bits in the string.

EXAMPLE Find the bitwise OR, bitwise AND, and bitwise XOR of the bit strings 01 1011 0110 and 11 0001 1101.

Solution: The bitwise OR, bitwise AND, and bitwise XOR are:

01 1011 0110

11 0001 1101

11 1011 1111 **Bitwise OR**

01 0001 0100 **Bitwise AND**

10 1010 1011 **Bitwise XOR**

Propositional Equivalences

Tautologies and contradictions

A compound proposition that is always true, regardless of the truth values of the individual propositions involved, is called a tautology.

Example: $p \vee \neg p$ is a tautology.

A compound proposition that is always false, regardless of the truth values of the individual propositions involved, is called a contradiction.

Example: $p \wedge \neg p$ is a contradiction.

A compound statement that is neither a tautology nor a contradiction is called a contingency.

Logical Equivalences

- Compound propositions that have the same truth values in all possible cases are called logically equivalent.
- The compound propositions p and q are called logically equivalent if $p \leftrightarrow q$ is a tautology. The notation $p \equiv q$ denotes that p and q are logically equivalent.

Examples of Tautology & De Morgan's Laws.					
	p	$\neg p$	$p \vee \neg p$	$p \wedge \neg p$	$\neg(p \wedge q) \equiv \neg p \vee \neg q$
Logical equivalence	T	F	T	F	
	F	T	T	F	$\neg(p \vee q) \equiv \neg p \wedge \neg q$

- The two algebraic expressions equal if they have the same value for each possible value of the input variables.
- For example, for all real numbers x , the left side and the right side have the same value.
- The two compound statements $x^2 - 1 = (x + 1)(x - 1)$ always share the same truth value. $p \equiv q$ means that $p \leftrightarrow q$ is a tautology.
- In these equivalences, T denotes the compound proposition that is always true and F denotes the compound proposition that is always false.

Logical Equivalences	
Equivalence	Name
$p \wedge T \equiv p$ $p \vee F \equiv p$	Identity laws
$p \vee T \equiv T$ $p \wedge F \equiv F$	Domination laws
$p \vee p \equiv p$ $p \wedge p \equiv p$	Idempotent laws
$\neg(\neg p) \equiv p$	Double negation law
$p \vee q \equiv q \vee p$ $p \wedge q \equiv q \wedge p$	Commutative laws
$(p \vee q) \vee r \equiv p \vee (q \vee r)$ $(p \wedge q) \wedge r \equiv p \wedge (q \wedge r)$	Associative laws
$p \vee (q \wedge r) \equiv (p \vee q) \wedge (p \vee r)$	Distributive laws

$p \wedge (q \vee r) \equiv (p \wedge q) \vee (p \wedge r)$	
$\neg(p \wedge q) \equiv \neg p \vee \neg q$ $\neg(p \vee q) \equiv \neg p \wedge \neg q$	De Morgan's laws
$p \vee (p \wedge q) \equiv p$ $p \wedge (p \vee q) \equiv p$	Absorption laws
$p \vee \neg p \equiv T$ $p \wedge \neg p \equiv F$	Negation laws

Logical Equivalences involving Conditional Statements.

$$\begin{aligned}
 p \rightarrow q &\equiv \neg p \vee q \\
 p \rightarrow q &\equiv \neg q \rightarrow \neg p \\
 p \vee q &\equiv \neg p \rightarrow q \\
 p \wedge q &\equiv \neg(p \rightarrow \neg q) \\
 \neg(p \rightarrow q) &\equiv p \wedge \neg q \\
 (p \rightarrow q) \wedge (p \rightarrow r) &\equiv p \rightarrow (q \wedge r) \\
 (p \rightarrow r) \wedge (q \rightarrow r) &\equiv (p \vee q) \rightarrow r \\
 (p \rightarrow q) \vee (p \rightarrow r) &\equiv p \rightarrow (q \vee r) \\
 (p \rightarrow r) \vee (q \rightarrow r) &\equiv (p \wedge q) \rightarrow r
 \end{aligned}$$

component propositions.

Example Show that $\neg(p \vee (\neg p \wedge q))$ and $\neg p \wedge \neg q$ are logically equivalent by developing a series of logical equivalences.

$$\begin{aligned}
 \text{Sol } \neg(p \vee (\neg p \wedge q)) &\equiv \neg p \wedge \neg(\neg p \wedge q) && \text{2nd De Morgan law} \\
 &\equiv \neg p \wedge [\neg(\neg p) \vee \neg q] && \text{1st De Morgan law} \\
 &\equiv \neg p \wedge (p \vee \neg q) && \text{double negation law} \\
 &\equiv (\neg p \wedge p) \vee (\neg p \wedge \neg q) && \text{2nd distributive law} \\
 &\equiv F \vee (\neg p \wedge \neg q) && \text{because } \neg p \wedge p \equiv F \\
 &\equiv (\neg p \wedge \neg q) \vee F && \text{commutative law for disjunction} \\
 &\equiv \neg p \wedge \neg q && \text{identity law for } F
 \end{aligned}$$

Normal Form

- Suppose, **A** (**P**₁, **P**₂, ... , **P**_n) is a statements formula where **P**₁, **P**₂, ..., **P**_n are the atomic variables if we consider all possible assignments of the truth value to **P**₁, **P**₂, ..., **P**_n and obtain the resulting truth values of the formula **A** then we get the truth table for **A**, such a truth table contains **2ⁿ** rows.
- The formula may have the truth value **T** for all possible assignments of the truth values to the variables **P**₁, **P**₂... **P**_n. In this case, **A** is called identically true or tautology.
- If **A** has the truth value **T** for at least one combination of truth values assigned to **P**₁, **P**₂... **P**_n then **A** is called Satisfiable.

DIWAKAR EDUCATION HUB

DISCRETE STRUCTURES AND OPTIMIZATION UNIT – 1 MCQS

As per updated syllabus

DIWAKAR EDUCATION HUB



THE LEARN WITH EXPERTIES

1. Which of the following statement is a proposition?

- a) Get me a glass of milkshake.
- b) God bless you!
- c) What is the time now?
- d) The only odd prime number is 2

Answer: d

Explanation: Only this statement has got the truth value which is false.

2. The truth value of '4+3=7 or 5 is not prime'.

- a) False
- b) True

Answer: b

Explanation: Compound statement with 'or' is true when either of the statement is true. Here the first part of the statement is true, hence the whole is true.

3. Which of the following option is true?

- a) If the Sun is a planet, elephants will fly
- b) $3 + 2 = 8$ if $5 - 2 = 7$
- c) $1 > 3$ and 3 is a positive integer
- d) $-2 > 3$ or 3 is a negative integer

Answer: a

Explanation: Hypothesis is false, thus the whole statement is true.

4. What is the value of x after this statement, assuming the initial value of x are 5?

'If x equals to one then $x=x+2$ else $x=0$ '.

- a) 1
- b) 3
- c) 0
- d) 2

Answer: c

Explanation: If condition is false so value decided according to else condition.

5. Let P: I am in Bangalore; Q: I love cricket; then $q \rightarrow p$ (q implies p) is?

- a) If I love cricket then I am in Bangalore

b) If I am in Bangalore then I love cricket

c) I am not in Bangalore

d) I love cricket

Answer: a

Explanation: Q is hypothesis and P is conclusion. So the compound statement will be if hypothesis then conclusion.

6. Which of the following bits is the negation of the bits "010110"?

111001

001001

101001

111111

Answer: c

Explanation: Flip each of the bits to get the negation of the required string.

7. Which of the following option is suitable, if A is "10110110", B is "11100000" and C is "10100000"?

a) $C=A$ or B

b) $C=\sim A$

c) $C=\sim B$

d) $C=A$ and B

Answer: d

Explanation: Output of and is 1 when both other inputs are one.

8. How many bits string of length 4 are possible such that they contain 2 ones and 2 zeroes?

a) 4

b) 2

c) 5

d) 6

Answer: d

Explanation: The strings are {0011, 0110, 1001, 1100, 1010 and 0101}.

9. If a bit string contains {0, 1} only, having length 5 has no more than 2 ones in it. Then how many such bit strings are possible?

a) 14

b) 12

c) 15

d) 16

Answer: d

Explanation: The total strings are 1(having no one in it) +5(having 1 one in it) +10(having 2 ones in it) = 16.

10. If A is "001100" and B is "010101" then what is the value of A (Ex-or) B?

a) 000000

b) 111111

c) 001101

d) 011001

Answer: d

Explanation: In Ex-or if both the inputs are same then output is 0 otherwise 1.

11. Let P and Q be statements, then $P \leftrightarrow Q$ is logically equivalent to _____

a) $P \leftrightarrow \sim Q$

b) $\sim P \leftrightarrow Q$

c) $\sim P \leftrightarrow \sim Q$

d) None of the mentioned

Answer: c

Explanation: Both of them have same truth table, hence they are equal.

12. What is the negation of the statement $A \rightarrow (B \vee (\text{or}) C)$?

a) $A \wedge \sim B \wedge \sim C$

b) $A \rightarrow B \rightarrow C$

c) $\sim A \wedge B \vee C$

d) None of the mentioned

Answer: a

Explanation: $A \rightarrow P$ is logically equivalent to $\sim A \vee P$.

13. The compound statement $A \rightarrow (A \rightarrow B)$ is false, then the truth values of A, B are respectively _____

a) T, T

b) F, T

c) T, F

d) F, F

Answer: c

Explanation: For implications to be false hypothesis should be true and conclusion should be false

14. The statement which is logically equivalent to $A \wedge (\text{and}) B$ is?

a) $A \rightarrow B$

b) $\sim A \wedge \sim B$

c) $A \wedge \sim B$

d) $\sim(A \rightarrow \sim B)$

Answer: d

Explanation: The truth table of both statements is same.

15. Let P, Q, R be true, false true, respectively, which of the following is true?

a) $P \wedge Q \wedge R$

b) $P \wedge \sim Q \wedge \sim R$

c) $Q \rightarrow (P \wedge R)$

d) $P \rightarrow (Q \wedge R)$

Answer: c

Explanation: Hypothesis is false, hence statement is true.

16. If there are 'M' switches in series numbered from 1, 2, ..., M. For circuit to be complete and bulb to glow which of the following is necessary?

a) $1 \wedge 2 \wedge 3 \wedge \dots \wedge M$ should be on.

b) $1 \wedge 2 \wedge 3 \wedge \dots \wedge M$ should be off.

c) $1 \vee 2 \vee 3 \vee \dots \vee M$ should be on.

d) None of the mentioned.

Answer: a

Explanation: All should be on in-order to complete the circuit.

17. Which of the following statements is the negation of the statements "4 is odd or -9 is positive"?

a) 4 is even or -9 is not negative.

b) 4 is odd or -9 is not negative.

c) 4 is even and -9 is negative.

d) 4 is odd and -9 is not negative.

Answer: c

Explanation: Using De Morgan's Law $\sim(A \vee B) \leftrightarrow \sim A \wedge \sim B$

18. Which of the following represents: $\sim A$ (negation of A) if A stands for “I like badminton but hate math’s”?

- a) I hate badminton and math’s
- b) I do not like badminton or math’s
- c) I dislike badminton but love math’s
- d) I hate badminton or like math’s

Answer: d

Explanation: De Morgan’s Law $\sim (A \wedge B) \leftrightarrow \sim A \vee \sim B$.

19. The compound statement $A \vee \sim(A \wedge B)$.

- a) True
- b) False

Answer: a

Explanation: Applying De-Morgan’s law we get $A \vee \sim A \equiv \text{Tautology}$.

20. Which of the following is De-Morgan’s law?

- a) $P \wedge (Q \vee R) \equiv (P \wedge Q) \vee (P \wedge R)$
- b) $\sim(P \wedge R) \equiv \sim P \vee \sim R, \sim(P \vee R) \equiv \sim P \wedge \sim R$
- c) $P \vee \sim P \equiv \text{True}, P \wedge \sim P \equiv \text{False}$
- d) None of the mentioned

Answer: b

Explanation: Definition of De–Morgan’s Law.

21. A compound proposition that is always _____ is called a tautology.

- a) True
- b) False

Answer: a

Explanation: Tautology is always true.

22. If A is any statement, then which of the following is a tautology?

- a) $A \wedge F$
- b) $A \vee F$
- c) $A \vee \sim A$
- d) $A \wedge T$

Answer: c

Explanation: $A \vee \sim A$ is always true.

23. A compound proposition that is neither a tautology nor a contradiction is called a _____

- a) Contingency
- b) Equivalence
- c) Condition
- d) Inference

Answer: a

Explanation: Definition of contingency.

24. $\neg (A \vee q) \wedge (A \wedge q)$ is a _____

- a) Tautology
- b) Contradiction
- c) Contingency
- d) None of the mentioned

Answer: b

Explanation: $\equiv (\neg A \wedge \neg q) \wedge (A \wedge q)$
 $\equiv (\neg A \wedge A) \wedge (\neg q \wedge q)$
 $\equiv F \wedge F \equiv F$.

25. The contra positive of $p \rightarrow q$ is the proposition of _____

- a) $\neg p \rightarrow \neg q$
- b) $\neg q \rightarrow \neg p$
- c) $q \rightarrow p$
- d) $\neg q \rightarrow p$

Answer: b

Explanation: Definition of contra positive.

26. The inverse of $p \rightarrow q$ is the proposition of _____

- a) $\neg p \rightarrow \neg q$
- b) $\neg q \rightarrow \neg p$
- c) $q \rightarrow p$
- d) $\neg q \rightarrow p$

Answer: a

Explanation: Definition of inverse.

27. The converse of $p \rightarrow q$ is the proposition of _____

- a) $\neg p \rightarrow \neg q$
- b) $\neg q \rightarrow \neg p$
- c) $q \rightarrow p$
- d) $\neg q \rightarrow p$

Answer: c

Explanation: Definition of converse.

28. What is the contra positive of the conditional statement? “The home team misses whenever it is drizzling?”

DIWAKAR EDUCATION HUB

**COMPUTER SYSTEM
ARCHITECTURE UNIT - 2**

As per updated syllabus

DIWAKAR EDUCATION HUB

THE LEARN WITH EXPERTIES

Digital Logic Circuits

The digital logic circuits are basic building blocks of the digital systems (digital computers). These digital logic circuits can be classified into two categories such as combinational logic circuits and sequential logic circuits. Before studying about the difference between combinational and sequential logic circuits, primarily, we must know what is combinational logic circuit and what are sequential logic circuits.

Digital Computers

A Digital computer can be considered as a digital system that performs various computational tasks.

The first electronic digital computer was developed in the late 1940s and was used primarily for numerical computations.

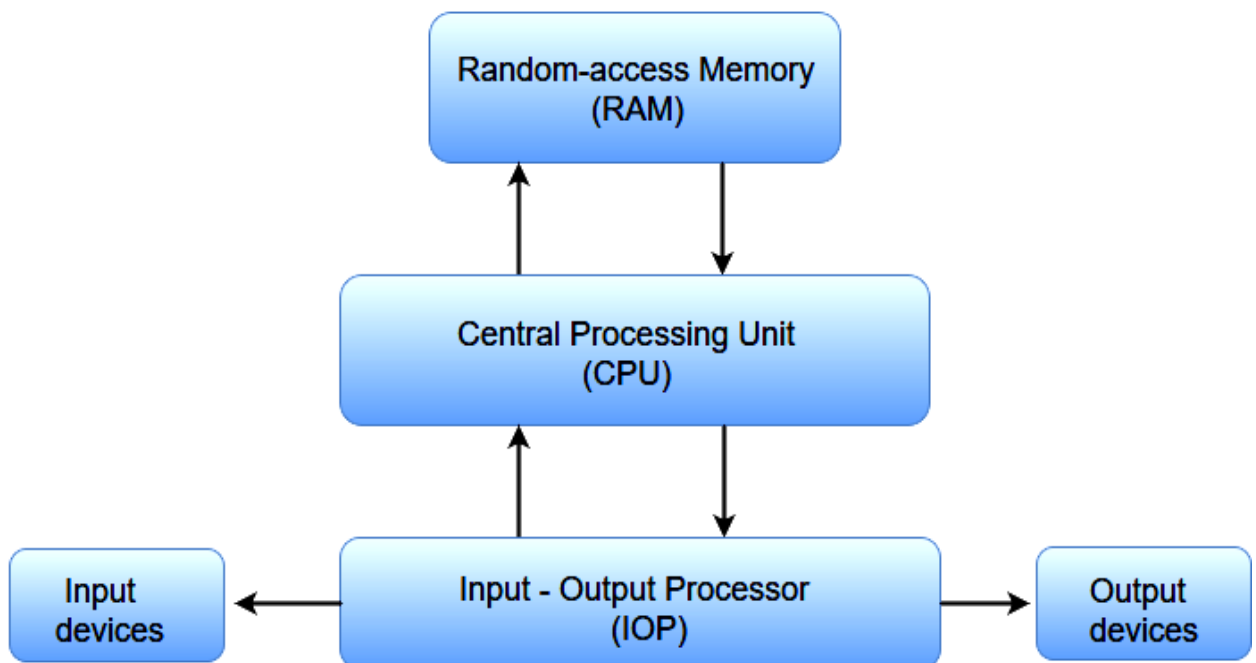
By convention, the digital computers use the binary number system, which has two digits: 0 and 1. A binary digit is called a bit.

A computer system is subdivided into two functional entities: Hardware and Software.

The hardware consists of all the electronic components and electromechanical devices that comprise the physical entity of the device.

The software of the computer consists of the instructions and data that the computer manipulates to perform various data-processing tasks.

Block diagram of a digital computer:



- The Central Processing Unit (CPU) contains an arithmetic and logic unit for manipulating data, a number of registers for storing data, and a control circuit for fetching and executing instructions.
- The memory unit of a digital computer contains storage for instructions and data.
- The Random Access Memory (RAM) for real-time processing of the data.
- The Input-Output devices for generating inputs from the user and displaying the final results to the user.

- The Input-Output devices connected to the computer include the keyboard, mouse, terminals, magnetic disk drives, and other communication devices.

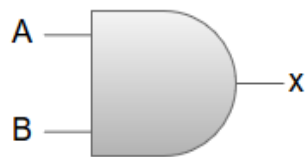
Logic Gates

- The logic gates are the main structural part of a digital system.
- Logic Gates are a block of hardware that produces signals of binary 1 or 0 when input logic requirements are satisfied.
- Each gate has a distinct graphic symbol, and its operation can be described by means of algebraic expressions.
- The seven basic logic gates includes: AND, OR, XOR, NOT, NAND, NOR, and XNOR.
- The relationship between the input-output binary variables for each gate can be represented in tabular form by a truth table.
- Each gate has one or two binary input variables designated by A and B and one binary output variable designated by x.

AND GATE:

The AND gate is an electronic circuit which gives a high output only if all its inputs are high. The AND operation is represented by a dot (.) sign.

AND Gate:



Algebraic Function: $x = AB$

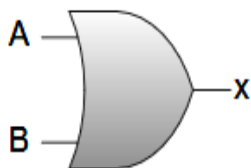
Truth Table:

A	B	x
0	0	0
0	1	0
1	0	0
1	1	1

OR GATE:

The OR gate is an electronic circuit which gives a high output if one or more of its inputs are high. The operation performed by an OR gate is represented by a plus (+) sign.

OR Gate:



Algebraic Function: $x = A + B$

Truth Table:

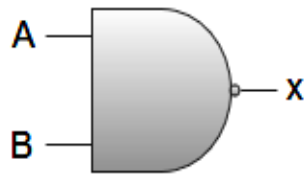
A	B	x
0	0	0
0	1	1
1	0	1
1	1	1

NOT GATE:

The NOT gate is an electronic circuit which produces an inverted version of the input at its output. It is also known as an **Inverter**.

NAND GATE:

The NOT-AND (NAND) gate which is equal to an AND gate followed by a NOT gate. The NAND gate gives a high output if any of the inputs are low. The NAND gate is represented by a AND gate with a small circle on the output. The small circle represents inversion.

NAND Gate:**Truth Table:**

Algebraic Function: $x = (AB)'$

A	B	x
0	0	1
0	1	1
1	0	1
1	1	0

NOR GATE:

The NOT-OR (NOR) gate which is equal to an OR gate followed by a NOT gate. The NOR gate gives a low output if any of the inputs are high. The NOR gate is represented by an OR gate with a small circle on the output. The small circle represents inversion.

NOR Gate:**Truth Table:**

Algebraic Function: $x = (A+B)'$

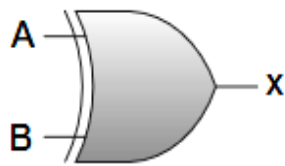
A	B	x
0	0	1
0	1	0
1	0	0
1	1	0

Exclusive-OR/ XOR GATE:

The 'Exclusive-OR' gate is a circuit which will give a high output if one of its inputs is high but not both of them. The XOR operation is represented by an encircled plus sign.

XOR Gate:

Truth Table:



Algebraic Function: $x = A \oplus B$

or

$$x = A'B + AB'$$

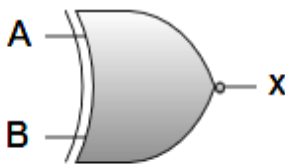
A	B	x
0	0	0
0	1	1
1	0	1
1	1	0

EXCLUSIVE-NOR/Equivalence GATE:

The 'Exclusive-NOR' gate is a circuit that does the inverse operation to the XOR gate. It will give a low output if one of its inputs is high but not both of them. The small circle represents inversion.

Exclusive-NOR Gate:

Truth Table:



Algebraic Function: $x = (A \oplus B)'$

or

$$x = A'B' + AB$$

A	B	x
0	0	1
0	1	0
1	0	0
1	1	1

Boolean Algebra

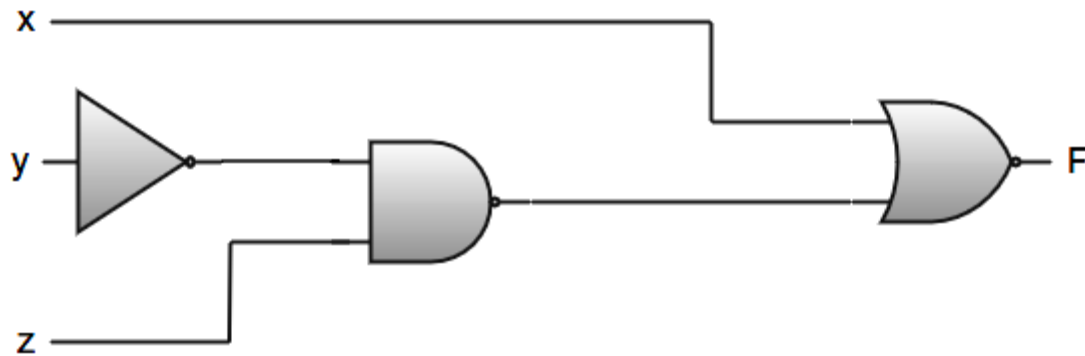
Boolean algebra can be considered as an algebra that deals with binary variables and logic operations. Boolean algebraic variables are designated by letters such as A, B, x, and y. The basic operations performed are AND, OR, and complement.

The Boolean algebraic functions are mostly expressed with binary variables, logic operation symbols, parentheses, and equal sign. For a given value of variables, the Boolean function can be either 1 or 0. For instance, consider the Boolean function:

$$F = x + y'z$$

The logic diagram for the Boolean function $F = x + y'z$ can be represented as:

$$F = x + y'z$$



- The Boolean function $F = x + y'z$ is transformed from an algebraic expression into a logic diagram composed of AND, OR, and inverter gates.
- Inverter at input 'y' generates its complement y' .
- There is an AND gate for the term $y'z$, and an OR gate is used to combine the two terms (x and $y'z$).
- The variables of the function are taken to be the inputs of the circuit, and the variable symbol of the function is taken as the output of the circuit.

Note: A truth table can represent the relationship between a function and its binary variables. To represent a function in a truth table, we need a list of the 2^n combinations of n binary variables.

The truth table for the Boolean function $F = x + y'z$ can be represented as:

$$F = x + y'z$$

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Laws of Boolean algebra

The basic Laws of Boolean Algebra can be stated as follows:

- Commutative Law states that the interchanging of the order of operands in a Boolean equation does not change its result. For example:
 1. OR operator $\rightarrow A + B = B + A$
 2. AND operator $\rightarrow A * B = B * A$
- Associative Law of multiplication states that the AND operation are done on two or more than two variables. For example:
$$A * (B * C) = (A * B) * C$$
- Distributive Law states that the multiplication of two variables and adding the result with a variable will result in the same value as multiplication of addition of the variable with individual variables. For example:
$$A + BC = (A + B) (A + C).$$
- Annulment law:
$$A.0 = 0$$
$$A + 1 = 1$$
- Identity law:
$$A.1 = A$$
$$A + 0 = A$$
- Idempotent law:
$$A + A = A$$
$$A.A = A$$
- Complement law:
$$A + A' = 1$$
$$A.A' = 0$$
- Double negation law:
$$((A)')' = A$$
- Absorption law:
$$A.(A+B) = A$$
$$A + AB = A$$

De Morgan's Law is also known as De Morgan's theorem, works depending on the concept of Duality. Duality states that interchanging the operators and variables in a function, such as replacing 0 with 1 and 1 with 0, AND operator with OR operator and OR operator with AND operator.

De Morgan stated 2 theorems, which will help us in solving the algebraic problems in digital electronics. The De Morgan's statements are:

1. "The negation of a conjunction is the disjunction of the negations", which means that the complement of the product of 2 variables is equal to the sum of the compliments of individual variables. For example, $(A.B)' = A' + B'$.
2. "The negation of disjunction is the conjunction of the negations", which means that compliment of the sum of two variables is equal to the product of the complement of each variable. For example, $(A + B)' = A'B'$.

Map Simplification

The Map method involves a simple, straightforward procedure for simplifying Boolean expressions.

Map simplification may be regarded as a pictorial arrangement of the truth table which allows an easy interpretation for choosing the minimum number of terms needed to express the function algebraically. The map method is also known as Karnaugh map or K-map.

Each combination of the variables in a truth table is called a mid-term.

Note: When expressed in a truth table a function of n variables will have 2^n min-terms, equivalent to the 2^n binary numbers obtained from n bits.

There are four min-terms in a two variable map. Therefore, the map consists of four squares, one for each min-term. The 0's and 1's marked for each row, and each column designates the values of variable x and y , respectively.

Two-variable map:

m0	m1
m3	m4

(a)

		y	
		0	1
x	0	$x'y'$	$x'y$
	1	xy'	xy

(b)

Representation of functions in the two-variable map:

		y	
		0	1
x	0		
	1		1

(a) xy

		y	
		0	1
x	0		1
	1	1	1

(b) $x+y$

Three variable map

There are eight min-terms in a three-variable map. Therefore, the map consists of eight squares.

Three variable map:

DIWAKAR EDUCATION HUB

COMPUTER SYSTEM ARCHITECTURE UNIT – 2 MCQs

As per updated syllabus

DIWAKAR EDUCATION HUB

THE LEARN WITH EXPERTIES

1. Any signed negative binary number is recognised by its _____

- a) MSB
- b) LSB
- c) Byte
- d) Nibble

Answer: a

Explanation: Any negative number is recognized by its MSB (Most Significant Bit). If it's 1, then it's negative, else if it's 0, then positive.

2. The parameter through which 16 distinct values can be represented is known as _____

- a) Bit
- b) Byte
- c) Word
- d) Nibble

Answer: c

Explanation: It can be represented up to 16 different values with the help of a Word. Nibble is a combination of four bits and Byte is a combination of 8 bits. It is "word" which is said to be a collection of 16-bits on most of the systems.

3. If the decimal number is a fraction then its binary equivalent is obtained by _____ the number continuously by 2.

- a) Dividing
- b) Multiplying
- c) Adding
- d) Subtracting

Answer: b

Explanation: On multiplying the decimal number continuously by 2, the binary equivalent is obtained by the collection of the integer part. However, if it's an integer, then it's binary equivalent is determined by dividing the number by 2 and collecting the remainders.

4. The representation of octal number

(532.2)₈ in decimal is _____

- a) (346.25)₁₀

b) (532.864)₁₀

c) (340.67)₁₀

d) (531.668)₁₀

Answer: a

Explanation: Octal to Decimal conversion is obtained by multiplying 8 to the power of base index along with the value at that index position.

$$(532.2)_8 = 5 * 8^2 + 3 * 8^1 + 2 * 8^0 + 2 * 8^{-1} = (346.25)_{10}$$

5. Which of the following is not a data type?

- a) Symbolic Data
- b) Alphanumeric Data
- c) Numeric Data
- d) Alphabetic Data

Answer: a

Explanation: Data types are of three basic types: Numeric, Alphabetic and Alphanumeric. Numeric Data consists of only numbers.

Alphabetic Data consists of only letters and a blank character and alphanumeric data consists of symbols.

6. _____ is the raw material used as input and _____ is the processed data obtained as output of data processing.

- a) Data, Instructions
- b) Instructions, Program
- c) Data, Program
- d) Program, Code

Answer: a

Explanation: Data can be assumed as a raw material which, in turns after processing gives the desired output in the form of instructions. Further, a set of ordered and meaningful instructions is known as a program.

7. Which of the following is not a characteristic of a computer?

- a) Diligence
- b) I.Q.
- c) Accuracy
- d) Versatility

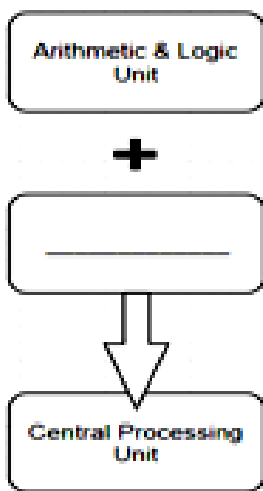
Answer: b

Explanation: The Computer system has no I.Q. of its own. It does only what it is programmed to do. It cannot take decisions of its own.

A computer is diligent because it can work continuously for hours without getting any errors or without getting grumbled.

The accuracy of a computer is consistently high and its level of accuracy depends on its design. A computer can perform any task if, it can be broken down into a series of logical steps. Therefore, a computer is versatile.

8. Fill in the blank in the diagram.



- a) Input Unit
- b) Memory Unit
- c) Control Unit
- d) I/O Unit

Answer: c

Explanation: The control unit manages and coordinates the operations of a computer system. The ALU is responsible for performing all the arithmetic and bitwise operations. Therefore, both these units combine to form the brain of the computer, which is the central processing unit.

9. The part of a processor which contains hardware necessary to perform all the operations required by a computer:

- a) Data path

b) Controller

c) Registers

d) Cache

Answer: a

Explanation: A processor is a part of the computer which does all the data manipulation and decision making. A processor comprises of:

A data path which contains the hardware necessary to perform all the operations. A controller tells the data path what needs to be done.

The registers act as intermediate storage for the data.

10. What does MAR stand for?

- a) Main Address Register
- b) Memory Access Register
- c) Main Accessible Register
- d) Memory Address Register

Answer: d

Explanation: MAR is a type of register which is responsible for the fetch operation. MAR is connected to the address bus and it specifies the address for the read and write operations.

11. If the control signals are generated by combinational logic, then they are generated by a type of _____ controlled unit.

- a) Micro programmed
- b) Software
- c) Logic
- d) Hardwired

Answer: d

Explanation: The main task of a control unit is to generate control signals. There are two main types of control units:

A hardwired control unit generates control signals by using combinational logic circuits and the Micro programmed control unit generates control signals by using some softwares.

12. Which is the simplest method of implementing hardwired control unit?

- a) State Table Method
- b) Delay Element Method
- c) Sequence Counter Method
- d) Using Circuits

Answer: a

Explanation: There are 3 ways of implementing hardwired control unit:

A state table is the simplest method in which a number of circuits are designed based on the cells in the table.

A delay element method consists of a flowchart drawn for the circuit. A D-flip flop is used as a delay element.

A sequence counter method used k-modulo counter as a replacement for k delay elements.

13. A set of microinstructions for a single machine instruction is called _____

- a) Program
- b) Command
- c) Micro program
- d) Micro command

Answer: c

Explanation: For every micro-operation, a set of microinstructions are written which indicate the control signals to be activated.

A set of microinstructions is a micro program. The address of the next microinstruction is given by a Micro-program counter.

14. Micro-program consists of a set of microinstructions which are strings of 0s and 1s.

- a) True
- b) False

Answer: a

Explanation: The computer understands only binary language. So, the micro-program should have instructions which are in the form of 0s and 1s. Each output line of

the micro-program corresponds to one control signal.

15. A decoder is required in case of a _____

- a) Vertical Microinstruction
- b) Horizontal Microinstruction
- c) Multilevel Microinstruction
- d) All types of microinstructions

Answer: a

Explanation: There are two types of microinstructions: Horizontal and Vertical. In a horizontal microinstruction, each bit represents a signal to be activated whereas, in case of vertical microinstruction bits are decoded and, the decoder then produces signals.

16. In boolean algebra, the OR operation is performed by which properties?

- a) Associative properties
- b) Commutative properties
- c) Distributive properties
- d) All of the Mentioned

Answer: d

Explanation: The expression for Associative property is given by $A+(B+C) = (A+B)+C$ & $A*(B*C) = (A*B)*C$.

The expression for Commutative property is given by $A+B = B+A$ & $A*B = B*A$.

The expression for Distributive property is given by $A+BC=(A+B)(A+C)$ & $A(B+C) = AB+AC$.

17. The expression for Absorption law is given by _____

- a) $A + AB = A$
- b) $A + AB = B$
- c) $AB + AA' = A$
- d) $A + B = B + A$

Answer: a

Explanation: The expression for Absorption Law is given by: $A+AB = A$.

Proof: $A + AB = A(1+B) = A$ (Since $1 + B = 1$ as per 1's Property).

DIWAKAR EDUCATION HUB

PROGRAMMING LANGUAGES AND COMPUTER GRAPHICS UNIT - 3

As per updated syllabus

DIWAKAR EDUCATION HUB

THE LEARN WITH EXPERTIES

Language Design

Language design techniques are the most important topic to design the Programming Language and to solve various types of problems in the discipline of Comp. Science and IT. The following are the major topics covered in Language Design.

1. Programming Language Concepts
2. PL-Paradigms and Models
3. Programming Environments
4. Translation process

Programming Language Concepts

Computer programming language, any of various languages for expressing a set of detailed instructions for a digital computer. Such instructions can be executed directly when they are in the computer manufacturer-specific numerical form known as machine language, after a simple substitution process when expressed in a corresponding assembly language, or after translation from some “higher-level” language. Although there are many computer languages, relatively few are widely used.

Machine and assembly languages are “low-level,” requiring a programmer to manage explicitly all of a computer’s idiosyncratic features of data storage and operation. In contrast, high-level languages shield a programmer from worrying about such considerations and provide a notation that is more easily written and read by programmers.

Language Types

Machine and assembly languages

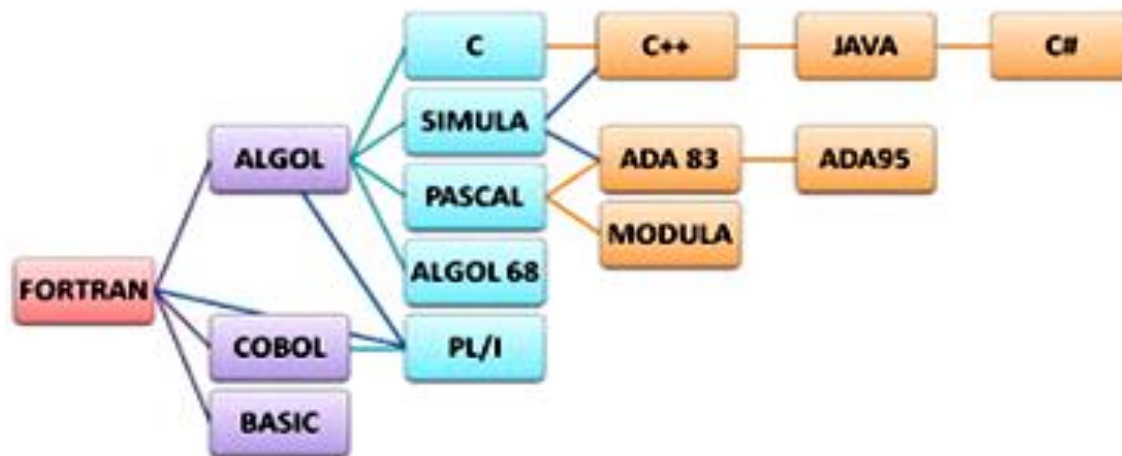
A machine language consists of the numeric codes for the operations that a particular computer can execute directly. The codes are strings of 0s and 1s, or binary digits (“bits”), which are frequently converted both from and to hexadecimal (base 16) for human viewing and modification. Machine language instructions typically use some bits to represent operations, such as addition, and some to represent operands, or perhaps the location of the next instruction. Machine language is difficult to read and write, since it does not resemble conventional mathematical notation or human language, and its codes vary from computer to computer.

Assembly language is one level above machine language. It uses short mnemonic codes for instructions and allows the programmer to introduce names for blocks of memory that hold data. One might thus write “add pay, total” instead of “0110101100101000” for an instruction that adds two numbers.

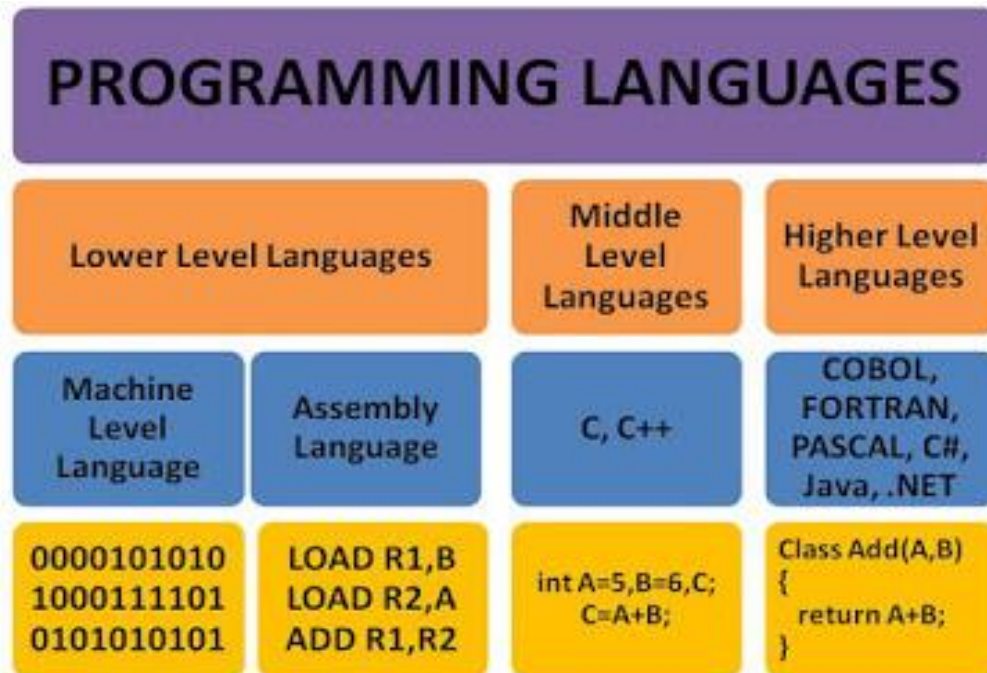
Assembly language is designed to be easily translated into machine language. Although blocks of data may be referred to by name instead of by their machine addresses, assembly language does not provide more sophisticated means of organizing complex information. Like machine language, assembly language requires detailed knowledge of internal computer architecture. It is useful when such details are important, as in programming a computer to interact with input/output devices (printers, scanners, storage devices, and so forth).

The built-in competence and defects of the various programming languages like FORTRAN, ALGOL, COBOL, C, C++, and JAVA. List of Differences between higher and lower/machine level languages.

Family tree of Major Programming Languages



Basic theories like abstract syntax, interpretation, stack, heap organization, compilation techniques, different types of 'type checking' and 'error checking' various for each Programming language.



Algorithmic languages

Algorithmic languages are designed to express mathematical or symbolic computations. They can express algebraic operations in notation similar to mathematics and allow the use of subprograms that package commonly used operations for reuse. They were the first high-level languages.

FORTRAN

The first important algorithmic language was FORTRAN (*formula translation*), designed in 1957 by an IBM team led by John Backus. It was intended for scientific computations with real numbers and collections of them organized as one- or multidimensional arrays. Its control

structures included conditional IF statements, repetitive loops (so-called DO loops), and a GOTO statement that allowed nonsequential execution of program code. FORTRAN made it convenient to have subprograms for common mathematical operations, and built libraries of them.

FORTRAN was also designed to translate into efficient machine language. It was immediately successful and continues to evolve.

ALGOL

ALGOL (*algorithmic language*) was designed by a committee of American and European computer scientists during 1958–60 for publishing algorithms, as well as for doing computations. Like LISP (described in the next section), ALGOL had recursive subprograms—procedures that could invoke themselves to solve a problem by reducing it to a smaller problem of the same kind. ALGOL introduced block structure, in which a program is composed of blocks that might contain both data and instructions and have the same structure as an entire program. Block structure became a powerful tool for building large programs out of small components.

ALGOL contributed a notation for describing the structure of a programming language, Backus–Naur Form, which in some variation became the standard tool for stating the syntax (grammar) of programming languages. ALGOL was widely used in Europe, and for many years it remained the language in which computer algorithms were published. Many important languages, such as Pascal and Ada (both described later), are its descendants.

LISP

LISP (*list processing*) was developed about 1960 by John McCarthy at the Massachusetts Institute of Technology (MIT) and was founded on the mathematical theory of recursive functions (in which a function appears in its own definition). A LISP program is a function applied to data, rather than being a sequence of procedural steps as in FORTRAN and ALGOL. LISP uses a very simple notation in which operations and their operands are given in a parenthesized list. For example, $(+ a (* b c))$ stands for $a + b * c$. Although this appears awkward, the notation works well for computers. LISP also uses the list structure to represent data, and, because programs and data use the same structure, it is easy for a LISP program to operate on other programs as data.

LISP became a common language for artificial intelligence (AI) programming, partly owing to the confluence of LISP and AI work at MIT and partly because AI programs capable of “learning” could be written in LISP as self-modifying programs. LISP has evolved through numerous dialects, such as Scheme and Common LISP.

C

The C programming language was developed in 1972 by Dennis Ritchie and Brian Kernighan at the AT&T Corporation for programming computer operating systems. Its capacity to structure data and programs through the composition of smaller units is comparable to that of ALGOL. It uses a compact notation and provides the programmer with the ability to operate with the addresses of data as well as with their values. This ability is important in systems programming, and C shares with assembly language the power to exploit all the features of a

computer's internal architecture. C, along with its descendant C++, remains one of the most common languages.

Business-oriented languages

COBOL

COBOL (*common business oriented language*) has been heavily used by businesses since its inception in 1959. A committee of computer manufacturers and users and U.S. government organizations established CODASYL (*Committee on Data Systems and Languages*) to develop and oversee the language standard in order to ensure its portability across diverse systems.

COBOL uses an English-like notation—novel when introduced. Business computations organize and manipulate large quantities of data, and COBOL introduced the record data structure for such tasks. A record clusters heterogeneous data such as a name, ID number, age, and address into a single unit. This contrasts with scientific languages, in which homogeneous arrays of numbers are common. Records are an important example of “chunking” data into a single object, and they appear in nearly all modern languages.

SQL

SQL (structured query language) is a language for specifying the organization of databases (collections of records). Databases organized with SQL are called relational because SQL provides the ability to query a database for information that falls in a given relation. For example, a query might be “find all records with both last_name Smith and city New York.” Commercial database programs commonly use a SQL-like language for their queries.

Education-oriented languages

BASIC

BASIC (beginner's all-purpose symbolic instruction code) was designed at Dartmouth College in the mid-1960s by John Kemeny and Thomas Kurtz. It was intended to be easy to learn by novices, particularly non-computer science majors, and to run well on a time-sharing computer with many users. It had simple data structures and notation and it was interpreted: a BASIC program was translated line-by-line and executed as it was translated, which made it easy to locate programming errors.

Its small size and simplicity also made BASIC a popular language for early personal computers. Its recent forms have adopted many of the data and control structures of other contemporary languages, which makes it more powerful but less convenient for beginners.

Pascal

About 1970 Niklaus Wirth of Switzerland designed Pascal to teach structured programming, which emphasized the orderly use of conditional and loop control structures without GOTO statements. Although Pascal resembled ALGOL in notation, it provided the ability to define data types with which to organize complex information, a feature beyond the capabilities of ALGOL as well as FORTRAN and COBOL. User-defined data types allowed the programmer to introduce names for complex data, which the language translator could then check for correct usage before running a program.

During the late 1970s and '80s, Pascal was one of the most widely used languages for programming instruction. It was available on nearly all computers, and, because of its familiarity, clarity, and security, it was used for production software as well as for education.

Logo

Logo originated in the late 1960s as a simplified LISP dialect for education; Seymour Papert and others used it at MIT to teach mathematical thinking to schoolchildren. It had a more conventional syntax than LISP and featured “turtle graphics,” a simple method for generating computer graphics. (The name came from an early project to program a turtlelike robot.) Turtle graphics used body-centred instructions, in which an object was moved around a screen by commands, such as “left 90” and “forward,” that specified actions relative to the current position and orientation of the object rather than in terms of a fixed framework. Together with recursive routines, this technique made it easy to program intricate and attractive patterns.

Hypertalk

Hypertalk was designed as “programming for the rest of us” by Bill Atkinson for Apple’s Macintosh. Using a simple English-like syntax, Hypertalk enabled anyone to combine text, graphics, and audio quickly into “linked stacks” that could be navigated by clicking with a mouse on standard buttons supplied by the program. Hypertalk was particularly popular among educators in the 1980s and early ’90s for classroom multimedia presentations. Although Hypertalk had many features of object-oriented languages (described in the next section), Apple did not develop it for other computer platforms and let it languish; as Apple’s market share declined in the 1990s, a new cross-platform way of displaying multimedia left Hypertalk all but obsolete (see the section World Wide Web display languages).

Object-oriented languages

Object-oriented languages help to manage complexity in large programs. Objects package data and the operations on them so that only the operations are publicly accessible and internal details of the data structures are hidden. This information hiding made large-scale programming easier by allowing a programmer to think about each part of the program in isolation. In addition, objects may be derived from more general ones, “inheriting” their capabilities. Such an object hierarchy made it possible to define specialized objects without repeating all that is in the more general ones.

Object-oriented programming began with the Simula language (1967), which added information hiding to ALGOL. Another influential object-oriented language was Smalltalk (1980), in which a program was a set of objects that interacted by sending messages to one another.

C++

The C++ language, developed by Bjarne Stroustrup at AT&T in the mid-1980s, extended C by adding objects to it while preserving the efficiency of C programs. It has been one of the most important languages for both education and industrial programming. Large parts of many operating systems, such as the Microsoft Corporation’s Windows 98, were written in C++.

Ada

Ada was named for Augusta Ada King, countess of Lovelace, who was an assistant to the 19th-century English inventor Charles Babbage, and is sometimes called the first computer programmer. Ada, the language, was developed in the early 1980s for the U.S. Department of Defense for large-scale programming. It combined Pascal-like notation with the ability to

package operations and data into independent modules. Its first form, Ada 83, was not fully object-oriented, but the subsequent Ada 95 provided objects and the ability to construct hierarchies of them. While no longer mandated for use in work for the Department of Defense, Ada remains an effective language for engineering large programs.

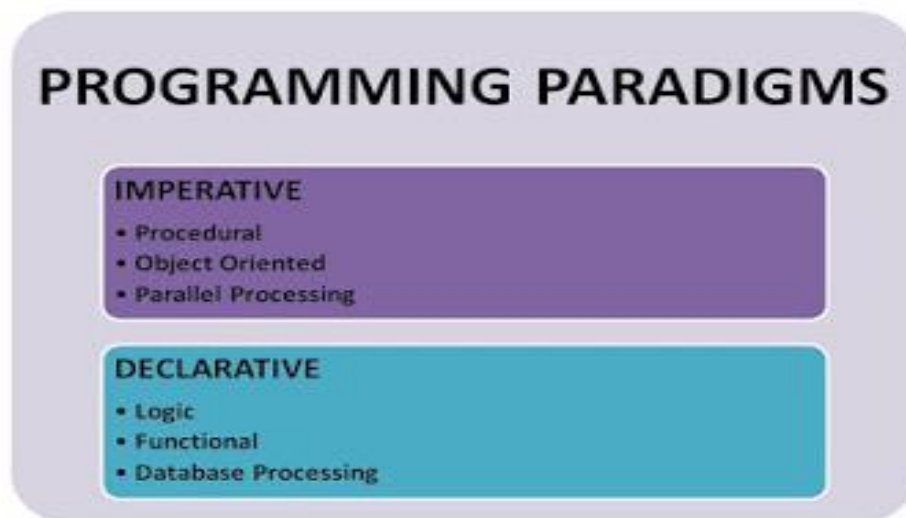
Java

In the early 1990s, Java was designed by Sun Microsystems, Inc., as a programming language for the World Wide Web (WWW). Although it resembled C++ in appearance, it was fully object-oriented. In particular, Java dispensed with lower-level features, including the ability to manipulate data addresses, a capability that is neither desirable nor useful in programs for distributed systems. In order to be portable, Java programs are translated by a Java Virtual Machine specific to each computer platform, which then executes the Java program. In addition to adding interactive capabilities to the Internet through Web “applets,” Java has been widely used for programming small and portable devices, such as mobile telephones.

PL-Paradigms and Models

A programming paradigm is a various system design of ideas that have been used to guide/implement the programming.

Paradigm can also be termed as method to solve some problem or do some task. Programming paradigm is an approach to solve problem using some programming language or also we can say it is a method to solve a problem using tools and techniques that are available to us following some approach. There are lots for programming language that are known but all of them need to follow some strategy when they are implemented and this methodology/strategy is paradigms. Apart from varieties of programming language there are lots of paradigms to fulfil each and every demand.



Following list of paradigms

Imperative: Explicit sequence [such as assignment statements] of statements that change the state of the computer. It is one of the oldest programming paradigm. It features close relation relation to machine architecture. It is based on Von Neumann architecture. It works by changing the program state through assignment statements. It performs step by step task by changing state. The main

focus is on how to achieve the goal. The paradigm consist of several statements and after execution of all the result is stored.

Advantage:

1. Very simple to implement
2. It contains loops, variables etc.

Disadvantage:

1. Complex problem cannot be solved
2. Less efficient and less productive
3. Parallel programming is not possible

Examples of **Imperative** programming paradigm:

C : developed by Dennis Ritchie and Ken Thompson

Fortan : developed by John Backus for IBM

Basic : developed by John G Kemeny and Thomas E Kurtz

```
// average of five number in C
int marks[5] = { 12, 32, 45, 13, 19 } int sum = 0;
float average = 0.0;
for (int i = 0; i < 5; i++) {
    sum = sum + marks[i];
}
average = sum / 5;
```

Imperative programming is divided into three broad categories: Procedural, OOP and parallel processing. These paradigms are as follows:

Procedural: Imperative programming with procedure calls something like the re-use of code.

This paradigm emphasizes on procedure in terms of under lying machine model. There is no difference in between procedural and imperative approach. It has the ability to reuse the code and it was boon at that time when it was in use because of its reusability.

Examples of **Procedural** programming paradigm:

C : developed by Dennis Ritchie and Ken Thompson

C++ : developed by Bjarne Stroustrup

Java : developed by James Gosling at Sun Microsystems

ColdFusion : developed by J J Allaire

Pascal : developed by Niklaus Wirth

Object-Oriented: By defining the objects that transfers messages to each objects or program. Objects have their own internal state with public interface calls [encapsulation].

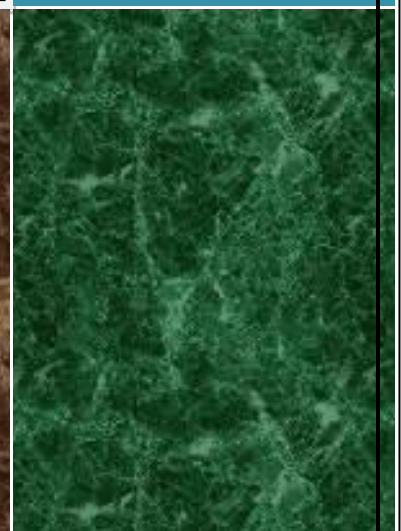
The program is written as a collection of classes and object which are meant for communication. The smallest and basic entity is object and all kind of computation is performed on the objects only. More emphasis is on data rather procedure. It can handle almost all kind of real life problems which are today in scenario.

DIWAKAR EDUCATION HUB

PROGRAMMING LANGUAGES AND COMPUTER GRAPHICS UNIT – 3 MCQS

As per updated syllabus

DIWAKAR EDUCATION HUB



THE LEARN WITH EXPERTIES

1) What is the 16-bit compiler allowable range for integer constants?

- a) -3.4e38 to 3.4e38
- b) -32767 to 32768
- c) -32668 to 32667
- d) -32768 to 32767

Answer: (d) -32768 to 32767

Explanation: In a 16-bit C compiler, we have 2 bytes to store the value.

- The range for signed integers is - 32768 to 32767.
- The range for unsigned integers is 0 to 65535.
- The range for unsigned character is 0 to 255.

2) Study the following program:

```
main()
{printf("javatpoint");
main();}
```

What will be the output of this program?

- a. Wrong statement
- b. It will keep on printing javatpoint
- c. It will Print javatpoint once
- d. None of the these

Answer: (b) It will keep on printing javatpoint

Explanation: In this program, the main function will call itself again and again. Therefore, it will continue to print javatpoint.

3) What is required in each C program?

- a. The program must have at least one function.
- b. The program does not require any function.
- c. Input data
- d. Output data

Answer: (a) The program must have at least one function.

Explanation: Any C program has at least one function, and even the most trivial

programs can specify additional functions. A function is a piece of code. In other words, it works like a sub-program.

4) What will this program print?

```
main()
{
    int i = 2;
    {
        int i = 4, j = 5;
        printf("%d %d", i, j);
    }
    printf("%d %d", i, j);
}
```

- a. 4525
- b. 2525
- c. 4545
- d. None of the these

Answer: (a) 4525

Explanation: In this program, it will first print the inner value of the function and then print the outer value of the function.

5) Which of the following comment is correct when a macro definition includes arguments?

- a. The opening parenthesis should immediately follow the macro name.
- b. There should be at least one blank between the macro name and the opening parenthesis.
- c. There should be only one blank between the macro name and the opening parenthesis.
- d. All the above comments are correct.

Answer: (a) The opening parenthesis should immediately follow the macro name.

6) What is a lint?

- a. C compiler
- b. Interactive debugger
- c. Analyzing tool
- d. C interpreter

Answer: (c) Analyzing tool

Explanation: Lint is an analyzing tool that analyzes the source code by suspicious constructions, stylistic errors, bugs, and flag programming errors. Lint is a compiler-like tool in which it parses the source files of C programming. It checks the syntactic accuracy of these files.

7) What is the output of this statement "printf("%d", (a++))"?

- a. The value of (a + 1)
- b. The current value of a
- c. Error message
- d. Garbage

Answer: (b) The current value of "a".

8) Study the following program:

```
main()
{
    char x [10], *ptr = x;
    scanf ("%s", x);
    change(&x[4]);
}
change(char a[])
{
    puts(a);
}
```

If abcdefg is the input, the output will be

- a. abcd
- b. abc
- c. efg
- d. Garbage

Answer: (c) efg

9) Study the following program:

```
main()
{
    int a = 1, b = 2, c = 3;
    printf("%d", a += (a += 3, 5, a))
}
```

What will be the output of this program?

- a. 6
- b. 9
- c. 12
- d. 8

Answer: (d) 8

Explanation: It is an effect of the comma operator.

a += (a += 3, 5, a)

It first evaluates to "a += 3" i.e. a = a + 3 then evaluate 5 and then evaluate "a".

Therefore, we will get the output is 4.

Then,

a += 4

It gives 8 as the output.

10) What does this declaration mean?

int x : 4;

- a. X is a four-digit integer.
- b. X cannot be greater than a four-digit integer.
- c. X is a four-bit integer.
- d. None of the these

Answer: (c) X is a four-bit integer.

Explanation: This means, "X" is a four bit integer.

11) Why is a macro used in place of a function?

- a. It reduces execution time.
- b. It reduces code size.
- c. It increases execution time.
- d. It increases code size.

Answer: (d) It reduces code size.

Explanation: Macro is used in place of a function because it reduces code size, and very efficient.

12) In the C language, the constant is defined _____.

- a. Before main
- b. After main

- c. Anywhere, but starting on a new line.
- d. None of the these.

Answer: (c) Anywhere, but starting on a new line.

Explanation: In the C language, the constant is defined anywhere, but starting on a new line.

13) How many times will the following loop execute?

```
for(j = 1; j <= 10; j = j-1)
```

- a. Forever
- b. Never
- c. 0
- d. 1

Answer: (a) Forever

14) A pointer is a memory address. Suppose the pointer variable has p address 1000, and that p is declared to have type `int*`, and an `int` is 4 bytes long. What address is represented by expression `p + 2`?

- a. 1002
- b. 1004
- c. 1006
- d. 1008

Answer: (d) 1008

15) What is the result after execution of the following code if a is 10, b is 5, and c is 10?

```
If ((a > b) && (a <= c))
```

```
    a = a + 1;
```

```
else
```

```
    c = c+1;
```

- a. a = 10, c = 10
- b. a = 11, c = 10
- c. a = 10, c = 11
- d. a = 11, c = 11

Answer: (b) a = 11, c = 10

16) Which one of the following is a loop construct that will always be executed once?

- a. for

- b. while
- c. switch
- d. do while

Answer: (d) do while

Explanation: The body of a loop is often executed at least once during the do-while loop. Once the body is performed, the condition is tested. If the condition is valid, it will execute the body of a loop; otherwise, control is transferred out of the loop.

17) Which of the following best describes the ordering of destructor calls for stack-resident objects in a routine?

- a. The first object created is the first object destroyed; last created is last destroyed.
- b. The first object destroyed is the last object destroyed; last created is first destroyed.
- c. Objects are destroyed in the order they appear in memory, the object with the lowest memory address is destroyed first.
- d. The order is undefined and may vary from compiler to compiler.

Answer: (b) The first object destroyed is the last object destroyed; last created is first destroyed.

18) How many characters can a string hold when declared as follows?

```
char name[20];
```

- a. 18
- b. 19
- c. 20
- d. None of the these

Answer: (b) 20

19) Directives are translated by the

- a. Pre-processor
- b. Compiler
- c. Linker

DIWAKAR EDUCATION HUB

**DATABASE MANAGEMENT
SYSTEMS UNIT – 4**

AS PER UPDATED SYLLABUS

DIWAKAR EDUCATION HUB

THE LEARN WITH EXPERTIES

Database Management System

Database Management System or DBMS in short refers to the technology of storing and retrieving users' data with utmost efficiency along with appropriate security measures.

Database is a collection of related data and data is a collection of facts and figures that can be processed to produce information.

Mostly data represents recordable facts. Data aids in producing information, which is based on facts. For example, if we have data about marks obtained by all students, we can then conclude about toppers and average marks.

A **database management system** stores data in such a way that it becomes easier to retrieve, manipulate, and produce information.

Database Management System or DBMS in short refers to the technology of storing and retrieving users' data with utmost efficiency along with appropriate security measures.

Characteristics

Traditionally, data was organized in file formats. DBMS was a new concept then, and all the research was done to make it overcome the deficiencies in traditional style of data management. A modern DBMS has the following characteristics –

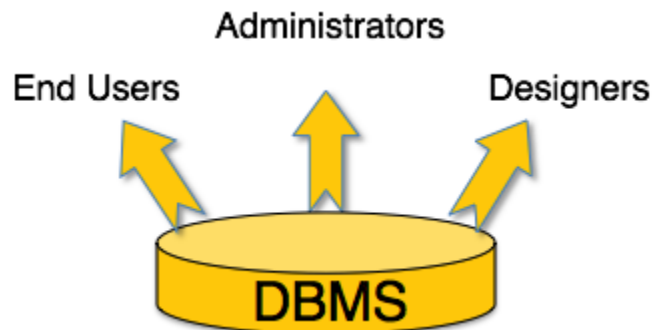
- **Real-world entity** – A modern DBMS is more realistic and uses real-world entities to design its architecture. It uses the behavior and attributes too. For example, a school database may use students as an entity and their age as an attribute.
- **Relation-based tables** – DBMS allows entities and relations among them to form tables. A user can understand the architecture of a database just by looking at the table names.
- **Isolation of data and application** – A database system is entirely different than its data. A database is an active entity, whereas data is said to be passive, on which the database works and organizes. DBMS also stores metadata, which is data about data, to ease its own process.
- **Less redundancy** – DBMS follows the rules of normalization, which splits a relation when any of its attributes is having redundancy in values. Normalization is a mathematically rich and scientific process that reduces data redundancy.
- **Consistency** – Consistency is a state where every relation in a database remains consistent. There exist methods and techniques, which can detect attempt of leaving database in inconsistent state. A DBMS can provide greater consistency as compared to earlier forms of data storing applications like file-processing systems.
- **Query Language** – DBMS is equipped with query language, which makes it more efficient to retrieve and manipulate data. A user can apply as many and as different filtering options as required to retrieve a set of data. Traditionally it was not possible where file-processing system was used.
- **ACID Properties** – DBMS follows the concepts of **A**tomicity, **C**onsistency, **I**solation, and **D**urability (normally shortened as ACID). These concepts are applied on transactions, which manipulate data in a database. ACID properties help the database stay healthy in multi-transactional environments and in case of failure.
- **Multiuser and Concurrent Access** – DBMS supports multi-user environment and allows them to access and manipulate data in parallel. Though there are restrictions on

transactions when users attempt to handle the same data item, but users are always unaware of them.

- **Multiple views** – DBMS offers multiple views for different users. A user who is in the Sales department will have a different view of database than a person working in the Production department. This feature enables the users to have a concentrate view of the database according to their requirements.
- **Security** – Features like multiple views offer security to some extent where users are unable to access data of other users and departments. DBMS offers methods to impose constraints while entering data into the database and retrieving the same at a later stage. DBMS offers many different levels of security features, which enables multiple users to have different views with different features. For example, a user in the Sales department cannot see the data that belongs to the Purchase department. Additionally, it can also be managed how much data of the Sales department should be displayed to the user. Since a DBMS is not saved on the disk as traditional file systems, it is very hard for miscreants to break the code.

Users

A typical DBMS has users with different rights and permissions who use it for different purposes. Some users retrieve data and some back it up. The users of a DBMS can be broadly categorized as follows –



- **Administrators** – Administrators maintain the DBMS and are responsible for administrating the database. They are responsible to look after its usage and by whom it should be used. They create access profiles for users and apply limitations to maintain isolation and force security. Administrators also look after DBMS resources like system license, required tools, and other software and hardware related maintenance.
- **Designers** – Designers are the group of people who actually work on the designing part of the database. They keep a close watch on what data should be kept and in what format. They identify and design the whole set of entities, relations, constraints, and views.
- **End Users** – End users are those who actually reap the benefits of having a DBMS. End users can range from simple viewers who pay attention to the logs or market rates to sophisticated users such as business analysts.

Applications of DBMS

Database is a collection of related data and data is a collection of facts and figures that can be processed to produce information.

Mostly data represents recordable facts. Data aids in producing information, which is based on facts. For example, if we have data about marks obtained by all students, we can then conclude about toppers and average marks.

A **database management system** stores data in such a way that it becomes easier to retrieve, manipulate, and produce information. Following are the important characteristics and applications of DBMS.

- **ACID Properties** – DBMS follows the concepts of **Atomicity**, **Consistency**, **Isolation**, and **Durability** (normally shortened as ACID). These concepts are applied on transactions, which manipulate data in a database. ACID properties help the database stay healthy in multi-transactional environments and in case of failure.
- **Multiuser and Concurrent Access** – DBMS supports multi-user environment and allows them to access and manipulate data in parallel. Though there are restrictions on transactions when users attempt to handle the same data item, but users are always unaware of them.
- **Multiple views** – DBMS offers multiple views for different users. A user who is in the Sales department will have a different view of database than a person working in the Production department. This feature enables the users to have a concentrate view of the database according to their requirements.
- **Security** – Features like multiple views offer security to some extent where users are unable to access data of other users and departments. DBMS offers methods to impose constraints while entering data into the database and retrieving the same at a later stage. DBMS offers many different levels of security features, which enables multiple users to have different views with different features. For example, a user in the Sales department cannot see the data that belongs to the Purchase department. Additionally, it can also be managed how much data of the Sales department should be displayed to the user. Since a DBMS is not saved on the disk as traditional file systems, it is very hard for miscreants to break the code.

DBMS - Architecture

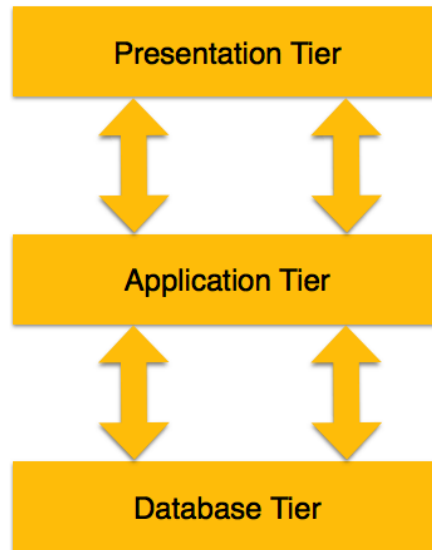
The design of a DBMS depends on its architecture. It can be centralized or decentralized or hierarchical. The architecture of a DBMS can be seen as either single tier or multi-tier. An n-tier architecture divides the whole system into related but independent **n** modules, which can be independently modified, altered, changed, or replaced.

In 1-tier architecture, the DBMS is the only entity where the user directly sits on the DBMS and uses it. Any changes done here will directly be done on the DBMS itself. It does not provide handy tools for end-users. Database designers and programmers normally prefer to use single-tier architecture.

If the architecture of DBMS is 2-tier, then it must have an application through which the DBMS can be accessed. Programmers use 2-tier architecture where they access the DBMS by means of an application. Here the application tier is entirely independent of the database in terms of operation, design, and programming.

3-tier Architecture

A 3-tier architecture separates its tiers from each other based on the complexity of the users and how they use the data present in the database. It is the most widely used architecture to design a DBMS.



- **Database (Data) Tier** – At this tier, the database resides along with its query processing languages. We also have the relations that define the data and their constraints at this level.
- **Application (Middle) Tier** – At this tier reside the application server and the programs that access the database. For a user, this application tier presents an abstracted view of the database. End-users are unaware of any existence of the database beyond the application. At the other end, the database tier is not aware of any other user beyond the application tier. Hence, the application layer sits in the middle and acts as a mediator between the end-user and the database.
- **User (Presentation) Tier** – End-users operate on this tier and they know nothing about any existence of the database beyond this layer. At this layer, multiple views of the database can be provided by the application. All views are generated by applications that reside in the application tier.

Multiple-tier database architecture is highly modifiable, as almost all its components are independent and can be changed independently.

Data Models

Data models define how the logical structure of a database is modeled. Data Models are fundamental entities to introduce abstraction in a DBMS. Data models define how data is connected to each other and how they are processed and stored inside the system.

The very first data model could be flat data-models, where all the data used are to be kept in the same plane. Earlier data models were not so scientific, hence they were prone to introduce lots of duplication and update anomalies.

Entity-Relationship Model

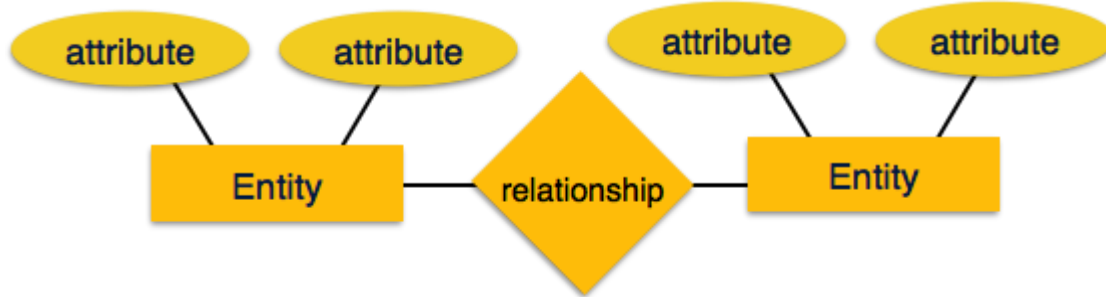
Entity-Relationship (ER) Model is based on the notion of real-world entities and relationships among them. While formulating real-world scenario into the database model, the ER Model creates entity set, relationship set, general attributes and constraints.

ER Model is best used for the conceptual design of a database.

ER Model is based on –

- **Entities** and their *attributes*.
- **Relationships** among entities.

These concepts are explained below.



- **Entity** – An entity in an ER Model is a real-world entity having properties called **attributes**. Every **attribute** is defined by its set of values called **domain**. For example, in a school database, a student is considered as an entity. Student has various attributes like name, age, class, etc.
- **Relationship** – The logical association among entities is called **relationship**. Relationships are mapped with entities in various ways. Mapping cardinalities define the number of association between two entities.

Mapping cardinalities –

- one to one
- one to many
- many to one
- many to many

Relational Model

The most popular data model in DBMS is the Relational Model. It is more scientific a model than others. This model is based on first-order predicate logic and defines a table as an **n-ary relation**.

attributes

column

SID	SName	SAge	SClass	SSection
1101	Alex	14	9	A
1102	Maria	15	9	A
1103	Maya	14	10	B
1104	Bob	14	9	A
1105	Newton	15	10	B

tuple

table (relation)

The diagram shows a table with 5 columns and 6 rows. The first row is the header. The second row is highlighted in orange. An arrow labeled 'attributes' points to the header row. An arrow labeled 'column' points to the first column. An arrow labeled 'tuple' points to the second row. A curved arrow labeled 'table (relation)' points to the entire table.

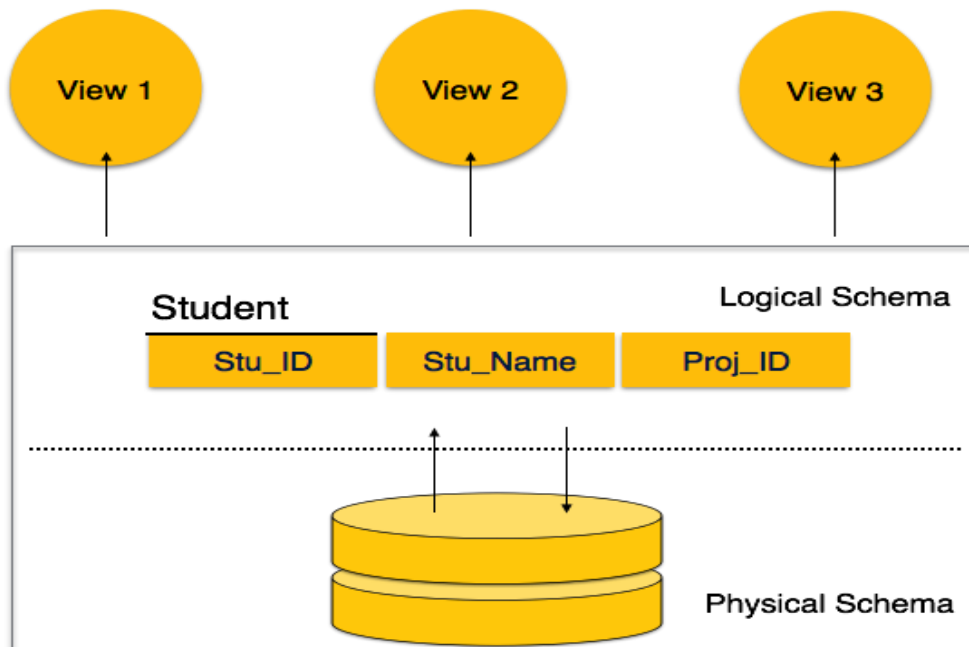
The main highlights of this model are –

- Data is stored in tables called **relations**.
- Relations can be normalized.
- In normalized relations, values saved are atomic values.
- Each row in a relation contains a unique value.
- Each column in a relation contains values from a same domain.

Data Schemas

A database schema is the skeleton structure that represents the logical view of the entire database. It defines how the data is organized and how the relations among them are associated. It formulates all the constraints that are to be applied on the data.

A database schema defines its entities and the relationship among them. It contains a descriptive detail of the database, which can be depicted by means of schema diagrams. It's the database designers who design the schema to help programmers understand the database and make it useful.



A database schema can be divided broadly into two categories –

- **Physical Database Schema** – This schema pertains to the actual storage of data and its form of storage like files, indices, etc. It defines how the data will be stored in a secondary storage.
- **Logical Database Schema** – This schema defines all the logical constraints that need to be applied on the data stored. It defines tables, views, and integrity constraints.

Database Instance

It is important that we distinguish these two terms individually. Database schema is the skeleton of database. It is designed when the database doesn't exist at all. Once the database is operational, it is very difficult to make any changes to it. A database schema does not contain any data or information.

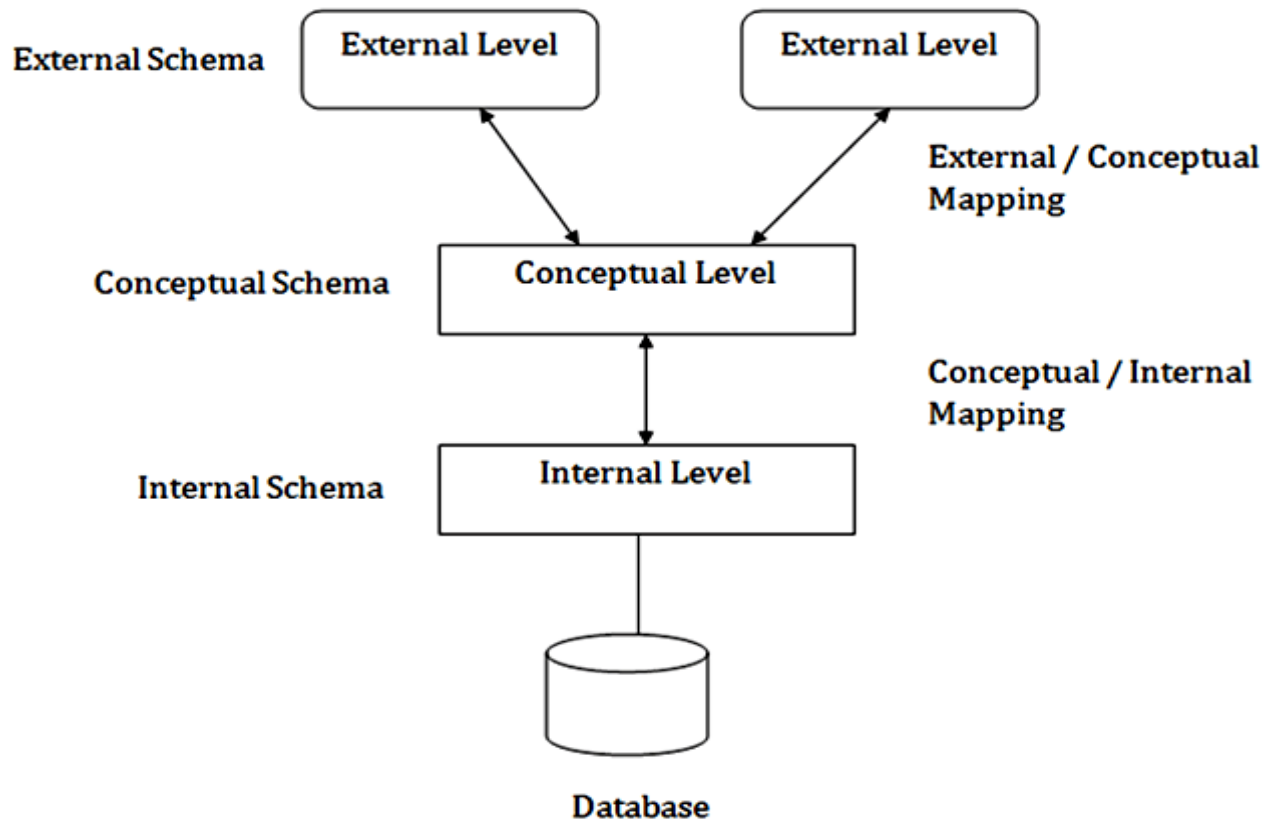
A database instance is a state of operational database with data at any given time. It contains a snapshot of the database. Database instances tend to change with time. A DBMS ensures

that its every instance (state) is in a valid state, by diligently following all the validations, constraints, and conditions that the database designers have imposed.

Three schema Architecture

- The three schema architecture is also called ANSI/SPARC architecture or three-level architecture.
- This framework is used to describe the structure of a specific database system.
- The three schema architecture is also used to separate the user applications and physical database.
- The three schema architecture contains three-levels. It breaks the database down into three different categories.

The three-schema architecture is as follows:



In the above diagram:

- It shows the DBMS architecture.
- Mapping is used to transform the request and response between various database levels of architecture.
- Mapping is not good for small DBMS because it takes more time.
- In External / Conceptual mapping, it is necessary to transform the request from external level to conceptual schema.
- In Conceptual / Internal mapping, DBMS transform the request from the conceptual to internal level.

1. Internal Level

- The internal level has an internal schema which describes the physical storage structure of the database.

DIWAKAR EDUCATION HUB

DATABASE MANAGEMENT SYSTEMS UNIT – 4 MCQs

AS PER UPDATED SYLLABUS

DIWAKAR EDUCATION HUB

THE LEARN WITH EXPERTIES

1. A relational database consists of a collection of

- a) Tables
- b) Fields
- c) Records
- d) Keys

Answer: a

Explanation: Fields are the column of the relation or tables. Records are each row in a relation. Keys are the constraints in a relation.

2. A _____ in a table represents a relationship among a set of values.

- a) Column
- b) Key
- c) Row
- d) Entry

Answer: c

Explanation: Column has only one set of values. Keys are constraints and row is one whole set of attributes. Entry is just a piece of data.

3. The term _____ is used to refer to a row.

- a) Attribute
- b) Tuple
- c) Field
- d) Instance

Answer: b

Explanation: Tuple is one entry of the relation with several attributes which are fields.

4. The term attribute refers to a _____ of a table.

- a) Record
- b) Column
- c) Tuple
- d) Key

Answer: b

Explanation: Attribute is a specific domain in the relation which has entries of all tuples.

5. For each attribute of a relation, there is a set of permitted values, called the _____ of that attribute.

- a) Domain
- b) Relation
- c) Set
- d) Schema

Answer: a

Explanation: The values of the attribute should be present in the domain. Domain is a set of values permitted.

6. Database _____ which is the logical design of the database, and the database _____ which is a snapshot of the data in the database at a given instant in time.

- a) Instance, Schema
- b) Relation, Schema
- c) Relation, Domain
- d) Schema, Instance

Answer: d

Explanation: Instance is an instance of time and schema is a representation.

7. Course(course_id,sec_id,semester)
Here the course_id,sec_id and semester are _____ and course is a _____

- a) Relations, Attribute
- b) Attributes, Relation
- c) Tuple, Relation
- d) Tuple, Attributes

Answer: b

Explanation: The relation course has a set of attributes course_id,sec_id,semester .

8. Department (dept name, building, budget) and Employee (employee_id, name, dept name, salary)
Here the dept_name attribute appears in both the relations. Here using common attributes in relation schema is one way of relating _____ relations.

- a) Attributes of common
- b) Tuple of common
- c) Tuple of distinct
- d) Attributes of distinct

Answer: c

Explanation: Here the relations are connected by the common attributes.

9. A domain is atomic if elements of the domain are considered to be _____ units.

- a) Different
- b) Indivisible
- c) Constant
- d) Divisible

Answer: b

10. The tuples of the relations can be of _____ order.

- a) Any
- b) Same
- c) Sorted
- d) Constant

Answer: a

Explanation: The values only count. The order of the tuples does not matter.

11. Which one of the following is a set of one or more attributes taken collectively to uniquely identify a record?

- a) Candidate key
- b) Sub key
- c) Super key
- d) Foreign key

Answer: c

Explanation: Super key is the superset of all the keys in a relation.

12. Consider attributes ID, CITY and NAME. Which one of this can be considered as a super key?

- a) NAME
- b) ID
- c) CITY
- d) CITY, ID

Answer: b

Explanation: Here the id is the only attribute which can be taken as a key. Other attributes are not uniquely identified.

13. The subset of a super key is a candidate key under what condition?

- a) No proper subset is a super key
- b) All subsets are super keys
- c) Subset is a super key
- d) Each subset is a super key

Answer: a

Explanation: The subset of a set cannot be the same set. Candidate key is a set from a super key which cannot be the whole of the super set.

14. A _____ is a property of the entire relation, rather than of the individual tuples in which each tuple is unique.

- a) Rows
- b) Key
- c) Attribute
- d) Fields

Answer: b

Explanation: Key is the constraint which specifies uniqueness.

15. Which one of the following attribute can be taken as a primary key?

- a) Name
- b) Street
- c) Id
- d) Department

Answer: c

Explanation: The attributes name, street and department can repeat for some tuples. But the id attribute has to be unique. So it forms a primary key.

16. Which one of the following cannot be taken as a primary key?

- a) Id
- b) Register number
- c) Dept_id
- d) Street

Answer: d

Explanation: Street is the only attribute which can occur more than once.

17. An attribute in a relation is a foreign key if the _____ key from one relation is used as an attribute in that relation.

- a) Candidate
- b) Primary
- c) Super
- d) Sub

Answer: b

Explanation: The primary key has to be referred in the other relation to form a foreign key in that relation.

18. The relation with the attribute which is the primary key is referenced in another relation. The relation which has the attribute as a primary key is called

- a) Referential relation
- b) Referencing relation
- c) Referenced relation
- d) Referred relation

Answer: b

19. The _____ is the one in which the primary key of one relation is used as a normal attribute in another relation.

- a) Referential relation
- b) Referencing relation
- c) Referenced relation
- d) Referred relation

Answer: c

20. A _____ integrity constraint requires that the values appearing in specified attributes of any tuple in the referencing relation also appear in specified attributes of at least one tuple in the referenced relation.

- a) Referential
- b) Referencing
- c) Specific
- d) Primary

Answer: a

Explanation: A relation, say r1, may include among its attributes the primary key of another relation, say r2. This attribute is called a foreign key from r1, referencing r2.

The relation r1 is also called the referencing relation of the foreign key dependency, and r2 is called the referenced relation of the foreign key.

21. Using which language can a user request information from a database?

- a) Query
- b) Relational
- c) Structural
- d) Compiler

Answer: a

Explanation: Query language is a method through which the database entries can be accessed.

22. Student(ID, name, dept name, tot_cred)
In this query which attributes form the primary key?

- a) Name
- b) Dept
- c) Tot_cred
- d) ID

Answer: d

Explanation: The attributes name, dept and tot_cred can have same values unlike ID.

23. Which one of the following is a procedural language?

- a) Domain relational calculus
- b) Tuple relational calculus
- c) Relational algebra
- d) Query language

Answer: c

Explanation: Domain and Tuple relational calculus are non-procedural language. Query language is a method through which database entries can be accessed.

24. The _____ operation allows the combining of two relations by merging pairs of tuples, one from each relation, into a single tuple.

- a) Select
- b) Join
- c) Union
- d) Intersection